

Material Design Using DFT

and Device Design by Energy Band Diagrams

TA Introduction Notes

Machiraju Karthikeya
PES University

शान्ति

1 Why Can't We Just Solve the Wave Equation?

Historical Insight

In 1926, Schrodinger wrote his wave equation. It describes electrons perfectly. It is exact.

For twenty years, physicists tried to use it directly on real materials. They failed. Not because the equation is wrong. Because it is too slow to compute.

Silicon has 14 electrons. To describe all of them together, the wavefunction needs 3 coordinates per electron. That is 42 numbers just to describe a single instant. For a real crystal with billions of atoms, it is hopeless.

So here is the core problem:

- We want to know the band gap, how fast electrons move, and whether a material is a conductor or insulator.
- All of these come from how electrons behave in the material.
- But tracking every single electron exactly is computationally impossible for anything real.

The three numbers we are chasing:

$$E_g \text{ (band gap), } m^* \text{ (effective mass), } \mu \text{ (mobility)}$$

All three live inside the electronic structure. You cannot get them without solving something quantum.

Key Takeaway

Density Functional Theory solves this by asking a different question.

Instead of: *where is every electron?*

It asks: *on average, how many electrons are at each point in space?*

That is the electron density $\rho(\mathbf{r})$. It is a function in 3 dimensions, not $3N$. That one change makes the whole thing tractable.

What Is the Electron Density?

The electron density tells you: at any point in space, how crowded are the electrons?

$$\rho(\mathbf{r}) = N \int |\Psi(\mathbf{r}, \mathbf{r}_2, \dots, \mathbf{r}_N)|^2 d\mathbf{r}_2 \cdots d\mathbf{r}_N$$

The formula looks scary. The idea is simple. $\rho(\mathbf{r})$ is just a 3D map of where the electrons tend to be.

Intuition

Think of a city at rush hour. Tracking every single person's exact location every second is impossible. But a population density map (people per square km) tells

you almost everything useful: where congestion happens, where ambulances are needed, where to build a school.

DFT uses the same idea for electrons. The density map replaces the individual tracking. The physics works out the same way.

2 The Two Theorems That Started DFT

Theorem 1: The Density Knows Everything

Historical Insight

In 1964, Hohenberg and Kohn wrote a proof that fits on two pages. They showed that the ground-state electron density $\rho(\mathbf{r})$ contains everything. If you know the density, you can figure out the potential the electrons feel. If you know the potential, you know the Hamiltonian. If you know the Hamiltonian, you know everything about the material.

Two pages. Nobel Prize in Chemistry in 1998. Not a bad trade.

The chain of reasoning:

$$\rho(\mathbf{r}) \longleftrightarrow V_{\text{ext}}(\mathbf{r}) \longleftrightarrow \hat{H} \longleftrightarrow \Psi_0 \longleftrightarrow \text{all properties}$$

The total energy written as a function of the density:

$$E[\rho] = T[\rho] + V_{\text{ext}}[\rho] + E_H[\rho] + E_{\text{xc}}[\rho]$$

- $T[\rho]$: kinetic energy of the electrons
- $V_{\text{ext}}[\rho]$: attraction to the nuclei. Computable exactly.
- $E_H[\rho]$: classical repulsion between electron clouds. Also exact.
- $E_{\text{xc}}[\rho]$: the quantum correlation stuff that nobody knows how to write exactly. This is the hard part.

Theorem 2: Nature Picks the Lowest Energy

The correct density is the one that gives the lowest total energy:

$$E_0 = \min_{\rho} E[\rho]$$

Intuition

This is just the variational principle. Nature is lazy. It always finds the lowest-energy arrangement. We use that same laziness computationally: try different densities, keep the one with the lowest energy, repeat until nothing changes.

3 The Kohn-Sham Trick: Making DFT Actually Work

Historical Insight

The HK theorems say the energy functional $E[\rho]$ exists. That is great. But the kinetic energy part $T[\rho]$ for interacting electrons? Nobody knows what it looks like.

In 1965, Kohn and Sham found a clever workaround. Their idea: stop trying to solve the real problem. Instead, invent a fake system of non-interacting electrons that happens to produce the same electron density as the real system. The fake system is easy to solve. The density you get out is still the real one. Sneaky. And it works.

The Kohn-Sham Equations

Mathematical Structure

Each fake electron gets its own equation:

$$\left[-\frac{\hbar^2}{2m} \nabla^2 + V_{\text{eff}}(\mathbf{r}) \right] \psi_i(\mathbf{r}) = \varepsilon_i \psi_i(\mathbf{r})$$

The potential each electron feels:

$$V_{\text{eff}}(\mathbf{r}) = V_{\text{ext}}(\mathbf{r}) + V_{\text{Hartree}}(\mathbf{r}) + V_{\text{xc}}(\mathbf{r})$$

The real density rebuilt from the fake orbitals:

$$\rho(\mathbf{r}) = \sum_i |\psi_i(\mathbf{r})|^2$$

What each term means in plain English:

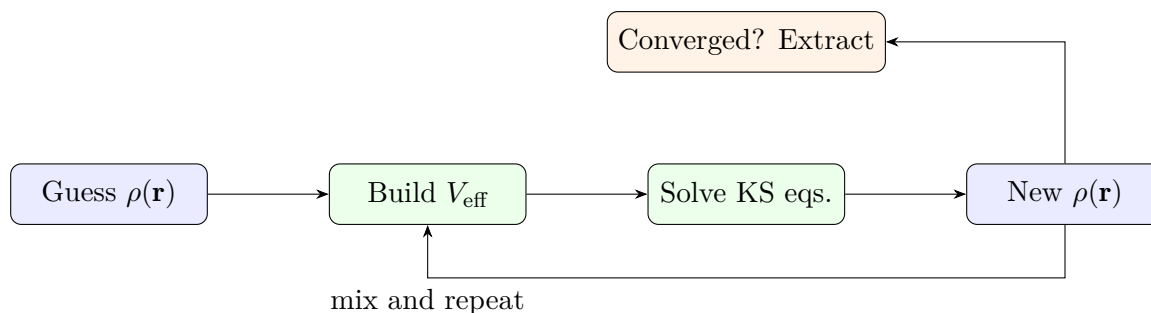
- V_{ext} : the pull from the atomic nuclei. This is just Coulomb's law. Exact.
- V_{Hartree} : the average push from all the other electrons. Also exact once you know ρ .
- V_{xc} : everything quantum and correlation-related that was left over. This is the only part we approximate.

Key Takeaway

V_{xc} is the only approximation in DFT. Every code, every paper, every calculation is really just a different way of guessing V_{xc} . Everything else is exact.

How DFT Runs: The SCF Loop

There is a chicken-and-egg problem. To compute V_{eff} we need ρ . To get ρ we need the orbitals. To get the orbitals we need V_{eff} . We solve it by going in circles until the answer stops changing:



This is called the SCF loop (self-consistent field). It stops when the density barely changes between steps.

Intuition

Think of a thermostat. You measure the room temperature (density). You adjust the heater (potential). You wait. Measure again. Stop when the temperature has stabilised.

Joke: an SCF that refuses to converge is like trying to lose weight by checking the scale every five minutes and eating based on the reading. You just keep oscillating. Nothing improves.

4 Choosing Your Approximation for V_{xc}

Historical Insight

John Perdew described the progression of approximations as Jacob's Ladder. Each rung is more accurate and more expensive. Your job as TA: help students pick the right rung for their question. Higher is not always better if you cannot afford the compute time.

LDA: The Simplest Option

Assumes the XC energy at any point in space is just whatever a uniform gas of electrons at that density would give:

$$E_{xc}^{LDA}[\rho] = \int \rho(\mathbf{r}) \varepsilon_{xc}(\rho(\mathbf{r})) d\mathbf{r}$$

Good: fast, works well for bulk metals and simple crystals. Bad: underestimates band gaps badly. Do not trust LDA gaps.

GGA: Also Looks at How the Density Changes

On top of the local density, also uses the gradient to see how fast the density is varying. Standard choice: PBE (1996):

$$E_{xc}^{GGA}[\rho] = \int \rho(\mathbf{r}) \varepsilon_{xc}(\rho(\mathbf{r}), |\nabla\rho(\mathbf{r})|) d\mathbf{r}$$

Good: better than LDA for most problems. The default for most calculations. Bad: still underestimates band gaps by 30 to 50 percent. This is a known, systematic error.

Key Takeaway

The band gap problem: silicon has a real band gap of 1.12 eV. PBE gives roughly 0.6 eV. This is not a bug. It is a known consequence of using an approximate V_{xc} . Every time you report a band gap, say which functional you used. Never just say “the DFT band gap.”

HSE06: The Hybrid Functional

Mixes in some exact Hartree-Fock exchange at short distances:

$$E_{xc}^{HSE} = \alpha E_x^{HF,SR} + (1 - \alpha) E_x^{GGA,SR} + E_x^{GGA,LR} + E_c^{GGA} \quad \alpha = 0.25$$

Good: much more accurate band gaps. The standard choice for semiconductor electronic structure. Bad: 5 to 10 times more expensive than GGA.

Intuition

Coffee analogy. LDA is instant coffee: very fast, not great. GGA is filter coffee: better. HSE06 is a proper espresso: excellent but takes longer. The GW approximation is a single-origin pour-over at a specialty cafe: the most accurate available, and you will mention it in every talk for a year, but you only run it once.

5 From Crystal Structure to Band Diagram: The Steps

Here is the actual sequence a student will follow. Every step has a reason. Skipping any one of them produces wrong results.

Step 1: Get the Crystal Structure

Get it from experiment (XRD data) or from a database like Materials Project or ICSD. You need: the space group, the lattice constants a , b , c , and where each atom sits in the unit cell (the Wyckoff positions).

Intuition

Always check Materials Project before running anything. If someone has already computed your material, their result is a free sanity check for your own calculation. Reinventing the wheel is fine for learning. Doing it in a published paper without checking is just slow.

Step 2: Relax the Structure

DFT is a zero-temperature theory. Experimental lattice constants come from room-temperature measurements. The two do not match. If you use experimental coordinates directly, the forces on atoms are nonzero and your results will be off.

Let the atoms move until the force on each one is very small:

$$F_i = -\frac{\partial E}{\partial \mathbf{R}_i} \longrightarrow \text{stop when } |F_i| < F_{\text{thresh}}$$

Key Takeaway

Most common beginner mistake: skip the relaxation and use the experimental geometry. The forces are large. The electronic structure is wrong. Relax first, every time.

Step 3: Run the SCF Calculation

With the relaxed structure, run the SCF loop on a grid of k-points to get the converged charge density. Three things to always check for convergence before trusting any number:

- The k-point grid: denser means more accurate and slower.
- The plane-wave energy cutoff E_{cut} : higher means more accurate and slower.
- The SCF convergence threshold: how small must the energy change between steps?

Step 4: Compute the Band Structure

Once SCF is done, fix the charge density and compute electron energies $\varepsilon_n(\mathbf{k})$ along a path through the high-symmetry points of the Brillouin zone. Common paths:

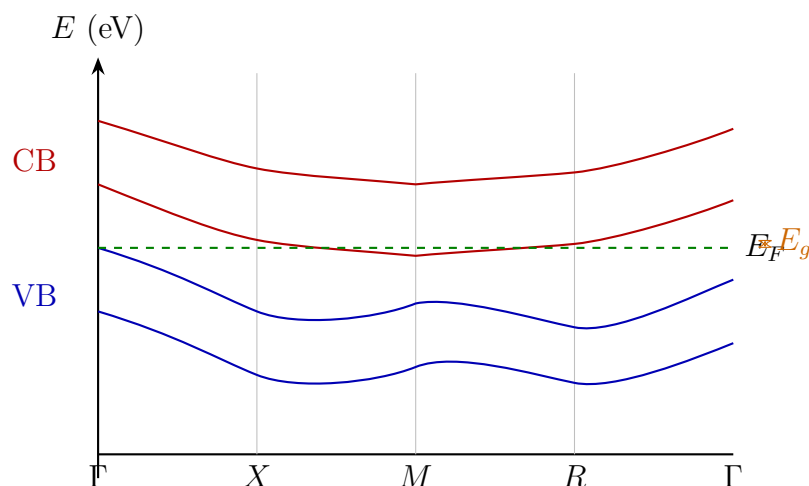
- FCC (Si, GaAs): $\Gamma \rightarrow X \rightarrow U|K \rightarrow \Gamma \rightarrow L \rightarrow W \rightarrow X$
- Hexagonal (GaN, MoS₂): $\Gamma \rightarrow M \rightarrow K \rightarrow \Gamma \rightarrow A$

Plot $\varepsilon_n(\mathbf{k})$ against $\mathbf{k}$. That is your band diagram.

6 How to Read a Band Diagram

The Basic Picture

A band diagram plots allowed electron energies on the vertical axis against crystal momentum $\mathbf{k}$ on the horizontal axis. Each curve is one electronic band.



- Valence bands (blue): full of electrons at absolute zero. These are the occupied states.
- Conduction bands (red): empty at absolute zero. Electrons need energy to reach these.
- Band gap E_g : a forbidden energy range. No electron can exist here.
- Fermi energy E_F : for a semiconductor at equilibrium, it sits in the middle of the gap.

Intuition

The parking garage: valence bands are the lower floors, completely full of parked cars (electrons). The conduction band is the rooftop level, completely empty. The band gap is the height of the ramp connecting them.

Metals have no ramp. Cars reach the roof freely. Electrons conduct.

Insulators have a 10-storey ramp. No car gets up. No current flows.

Semiconductors have a 1 to 3-storey ramp. At room temperature, a few cars get pushed up by thermal energy. That small number on the roof is what carries the current.

Direct vs Indirect Gap: Why It Matters for LEDs

- Direct gap: the top of the valence band and the bottom of the conduction band are at the same $\mathbf{k}$ -point. One photon can move an electron across the gap. Efficient light emitter. GaAs, GaN, and InP are all direct-gap materials. They are used in every LED and laser diode.
- Indirect gap: the band edges are at different $\mathbf{k}$ -points. Moving an electron across the gap requires both a photon (for energy) and a phonon (to change momentum). This is a three-body process and it rarely happens. Silicon and germanium are both indirect-gap materials and are poor light emitters.

Historical Insight

This is why silicon solar cells are 100 to 300 microns thick while GaAs cells work at 1 micron. Silicon absorbs sunlight weakly because of its indirect gap. You need much more material to capture the same light.

And this is why you have never seen a silicon LED in any product.

Silicon tried to be a light source. The phonon said no.

Effective Mass: How Fast Do Carriers Move?

Near the top of a valence band or bottom of a conduction band, the dispersion looks parabolic:

$$E(\mathbf{k}) = E_0 + \frac{\hbar^2 k^2}{2m^*}$$

The effective mass m^* is just the curvature of that parabola:

$$\frac{1}{m^*} = \frac{1}{\hbar^2} \frac{d^2 E}{dk^2}$$

Steep narrow parabola: small m^* , fast carriers. Broad flat parabola: large m^* , slow carriers.

Key Takeaway

InAs has $m^* \approx 0.023 m_e$. Its electrons are about 40 times lighter than free electrons. This is why III-V semiconductors are used in high-frequency electronics. The shape of the band tells you the speed of the device.

7 Heterostructures: When Two Semiconductors Meet

When you stack two different semiconductors, their band edges have to line up somehow at the interface. How they line up determines what the device actually does.

The Three Types

- **Type I (straddling)**: both the conduction band edge and the valence band edge of one material sit inside the gap of the other. Both electrons and holes end up trapped in the same layer. They meet and recombine easily. Good for lasers and LEDs. Example: GaAs/AlGaAs quantum wells.
- **Type II (staggered)**: the band edges are offset so electrons go to one side and holes go to the other. They are physically separated. Charge separation. Good for solar cells and photodetectors.
- **Type III (broken gap)**: the bands overlap completely. Leads to tunnelling behaviour. Used in tunnel transistors. Example: InAs/GaSb.

Intuition

Type I is a swimming pool. Everyone ends up in the water together (recombination happens easily).

Type II is an apartment where the kids (electrons) sleep upstairs and the parents (holes) sleep downstairs. They are separated and rarely meet.

Type III is when the floors physically overlap. Sounds impossible. Quantum mechanics is fine with it.

Computing Band Offsets with DFT

Each DFT calculation has its own internal energy zero, so you cannot directly subtract band energies from two separate runs. The standard method:

1. Run DFT on material A. Reference the VBM to a core level or the vacuum level.
2. Run DFT on material B. Reference the VBM to the same level.

3. Valence band offset: $\Delta E_v = E_v^A - E_v^B$
4. Conduction band offset: $\Delta E_c = \Delta E_v + (E_g^B - E_g^A)$

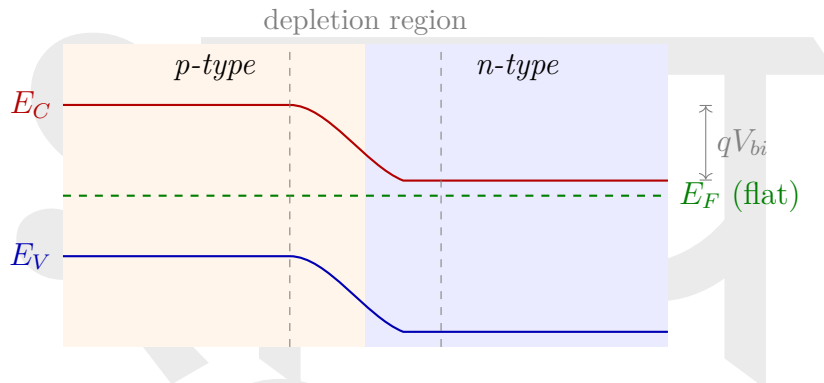
Key Takeaway

The sign of the offset determines the device type. GGA offsets carry errors of 0.1 to 0.3 eV. For any real device design, use HSE06 for the final numbers.

8 The p-n Junction Band Diagram

The p-n junction is the foundation of almost every semiconductor device. Reading its band diagram fluently is a core skill.

At Equilibrium: The Flat Fermi Level Rule



Key Takeaway

At equilibrium, the Fermi level is flat across the entire device. This is the definition of thermal equilibrium. If a student draws a p-n junction and the Fermi level is not flat, the diagram is wrong.

The built-in potential:

$$V_{bi} = \frac{kT}{q} \ln \left(\frac{N_A N_D}{n_i^2} \right)$$

Under Bias

When you apply a voltage, the single Fermi level splits into two quasi-Fermi levels:

$$F_n - F_p = qV$$

- Forward bias (positive voltage on the p-side): the barrier gets smaller. Current flows.
- Reverse bias: the barrier gets bigger. Almost no current.

Intuition

The flat Fermi level is the master key for reading any device band diagram. Whenever something drives the system out of equilibrium (a voltage, light shining on it, heat), the Fermi level splits into two. The gap between those two levels tells you exactly how far from equilibrium you are. Students who understand this one rule can read any band diagram ever drawn.

9 Things Students Get Wrong

Historical Insight

Every generation of students makes the same mistakes. This is not a reflection on the students. The ideas are genuinely tricky.

Your job as TA is not to stop students from making mistakes. Your job is to help them find the mistake, understand what went wrong, and move on faster than they would by themselves.

Mistake 1: Trusting the DFT Band Gap

PBE gives silicon about 0.6 eV. The real value is 1.12 eV. This is a known, systematic limitation. Use HSE06 or GW for final band gap numbers. Always say which functional you used.

Mistake 2: Skipping the Relaxation

DFT forces at experimental atomic positions are nonzero. The atoms need to be moved to where DFT wants them. Relax the structure first, every time.

Mistake 3: Too Few k-Points

Band gap, total energy, and DOS all depend on how dense the k-point grid is. A coarse grid gives wrong numbers. Run a convergence test on the k-mesh before reporting anything.

Mistake 4: Forgetting Spin-Orbit Coupling

For heavy elements ($Z > 50$): Bi, Pb, Te, and anything topological. Bi_2Se_3 without spin-orbit coupling looks completely different from Bi_2Se_3 with it included. Always include SOC for heavy atoms.

Mistake 5: Misreading the Band Gap Type

Students read off the smallest vertical gap on the plot without checking whether the top of the valence band and the bottom of the conduction band are at the same k-point. Check the k-coordinates explicitly before saying whether the gap is direct or indirect.

Key Takeaway

When a student's result looks wrong, ask these four questions in order:

1. Did you relax the structure?
2. Did you converge the k-points and the cutoff energy?
3. Which XC functional did you use?
4. Did you include spin-orbit coupling?

These four questions fix nine out of ten problems.

10 Python Simulations: Start Here Before QE

Historical Insight

Before Quantum ESPRESSO, before input files, before pseudopotentials, start here. These four Python scripts need only numpy and matplotlib. They run on a laptop in minutes. They build the intuition that makes everything in Quantum ESPRESSO make sense later.

Write the code. Run it. Look at the output. Change one number. Run again. That is the whole method.

What You Need

Install in one line

```
pip install numpy matplotlib scipy
```

No QE. No supercomputer. No pseudopotentials. Just Python and three packages.

Simulation 1: Electron in a Box

What is this about? An electron trapped inside a box can only have certain allowed energies. This is quantum confinement. It is what happens inside a quantum dot, a nanowire, and at the gate interface of every modern transistor. We solve the Schrodinger equation numerically and watch what happens when we shrink the box.

What to look for: Run with $L = 1.0$, then change to $L = 0.3$. Watch the energy levels shoot up. The smaller the box, the higher the energies. This is why a smaller quantum dot emits higher-energy (bluer) light.

sim1_particle_in_box.py

```
import numpy as np
import matplotlib.pyplot as plt

L = 1.0      # Box width in nm. Try changing this to 0.3!
```

```

N = 500          # Grid points
x = np.linspace(0, L, N)
dx = L / N
h2over2m = 3.81 # hbar^2/2m in eV*nm^2 for a real electron

# Build the kinetic energy matrix using finite differences
diag = 2.0 * np.ones(N)
off_diag = -1.0 * np.ones(N - 1)
H = (h2over2m / dx**2) * (
    np.diag(diag) + np.diag(off_diag, 1) + np.diag(off_diag, -1)
)

# Solve: energies (eigenvalues) and wavefunctions (eigenvectors)
energies, wavefunctions = np.linalg.eigh(H)

# Plot 1: wavefunctions sitting at their energy levels
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
ax = axes[0]
colors = ['royalblue', 'tomato', 'seagreen', 'darkorange']
for n in range(4):
    psi = wavefunctions[:, n] / np.max(np.abs(wavefunctions[:, n]))
    ax.plot(x, psi + energies[n], color=colors[n], lw=2, label=f'n={n+1}')
    ax.axhline(energies[n], color=colors[n], lw=0.8, ls='--', alpha=0.5)
ax.set_xlabel('Position (nm)')
ax.set_ylabel('Energy (eV)')
ax.set_title(f'Wavefunctions in a {L} nm box')
ax.legend()

# Plot 2: ground state energy vs box size (quantum confinement)
ax2 = axes[1]
box_sizes = np.linspace(0.3, 3.0, 200)
E1 = []
for l in box_sizes:
    dx1 = l / N
    H1 = (h2over2m / dx1**2) * (
        np.diag(2.0*np.ones(N)) +
        np.diag(-np.ones(N-1), 1) +
        np.diag(-np.ones(N-1), -1)
    )
    e, _ = np.linalg.eigh(H1)
    E1.append(e[0])
ax2.plot(box_sizes, E1, color='royalblue', lw=2)
ax2.set_xlabel('Box width L (nm)')
ax2.set_ylabel('Ground state energy (eV)')
ax2.set_title('Smaller box = higher energy')

plt.tight_layout()
plt.savefig('sim1_particle_in_box.pdf', dpi=150)
plt.show()

print("First 4 energy levels:")
for n, e in enumerate(energies[:4]):
    print(f"  n={n+1}: {e:.3f} eV")

```

Key Takeaway

Run this with $L = 1.0$. Then change to $L = 0.3$ and run again. Watch the energy levels go up. A 2 nm quantum dot glows blue. A 5 nm dot glows red. The only difference is the box size. Energies go as $1/L^2$, so halving the box quadruples the energies.

Simulation 2: Where Band Gaps Come From

What is this about? Why do some materials have band gaps at all? It is because the periodic arrangement of atoms in a crystal creates a periodic potential that electrons

feel as they move through. That potential opens up gaps at certain energies. This script shows you the whole mechanism in about 20 lines.

What to look for: Set $V_0 = 0$ and run. You get a metal: no gap. Increase V_0 step by step. Watch the gaps open up at the zone boundary. The gap size is directly controlled by the strength of the periodic potential.

sim2_band_gaps.py

```
import numpy as np
import matplotlib.pyplot as plt

V0 = 2.0      # Periodic potential in eV. Try 0, then 1, then 4, then 8.
a = 0.543     # Lattice constant in nm (silicon)
k = np.linspace(-np.pi/a, np.pi/a, 400)
h2over2m = 3.81 # eV*nm^2

# Reciprocal lattice vectors:  $G = n * 2\pi/a$ 
G = np.array([-2, -1, 0, 1, 2]) * (2 * np.pi / a)

all_bands = []
for ki in k:
    H = np.zeros((len(G), len(G)))
    for i, Gi in enumerate(G):
        H[i, i] = h2over2m * (ki + Gi)**2 # kinetic energy on diagonal
        for j in range(len(G)):
            if abs(i - j) == 1:
                H[i, j] = V0 / 2 # periodic coupling off-diagonal
    all_bands.append(np.linalg.eigvalsh(H))

all_bands = np.array(all_bands)

fig, ax = plt.subplots(figsize=(7, 6))
colors = ['royalblue', 'tomato', 'seagreen', 'darkorange']
for n in range(4):
    ax.plot(k, all_bands[:, n], color=colors[n], lw=2, label=f'Band {n+1}')
    ax.axvline(np.pi/a, color='gray', lw=0.8, ls='--', label='Zone boundary')
    ax.axvline(-np.pi/a, color='gray', lw=0.8, ls='--')
    ax.set_xlabel(r' $k$  (nm-1)')
    ax.set_ylabel('Energy (eV)')
    ax.set_title(f'Nearly-free electron bands (V0 = {V0} eV)')
    ax.set_ylim(0, 60)
    ax.legend()
plt.tight_layout()
plt.savefig('sim2_band_gaps.pdf', dpi=150)
plt.show()

mid = len(k) // 2
print(f"Gap at zone boundary: {all_bands[mid, 2] - all_bands[mid, 1]:.2f} eV")
print(f"(Controlled by V0 = {V0} eV)")
```

Intuition

Set $V_0 = 0$ and run. No gap: this looks like a metal. Now increase V_0 to 4 and run again. The gaps grow. The whole reason semiconductors exist is captured in this one script: the periodic lattice creates a periodic potential, and that potential opens gaps. Students who run this simulation understand something that students who only read about it often do not.

Simulation 3: Band Structure Plotter

What is this about? A complete band structure plotter that works right now using fake data. When students have real output from Quantum ESPRESSO, they swap in the

real numbers at the top. Everything else stays the same.

What to look for: Learn how the plot is built: k-points on the x-axis, energy on the y-axis, Fermi level at zero, vertical lines at high-symmetry points. Get comfortable with this layout before you run your first real DFT calculation.

sim3_band_plotter.py

```
import numpy as np
import matplotlib.pyplot as plt

# Replace this block with your real QE data when you have it.
# Everything below stays the same regardless of where the data comes from.
n_kpts = 100
n_bands = 8
k = np.linspace(0, 1, n_kpts)
bands = np.zeros((n_kpts, n_bands))
for n in range(n_bands):
    phase = n * np.pi / 4
    bands[:, n] = (n - 3.5) * 2.0 + 1.5 * np.cos(2 * np.pi * k + phase)
bands -= bands[:, 3].max() # set valence band maximum to 0 eV

# High-symmetry point positions and labels
hs_pos = [0.0, 0.25, 0.5, 0.75, 1.0]
hs_labels = [r'$\Gamma$', 'X', 'M', 'R', r'$\Gamma$']

fig, ax = plt.subplots(figsize=(6, 7))
for n in range(n_bands):
    ax.plot(k, bands[:, n], color='royalblue', lw=1.4)
ax.axhline(0, color='black', lw=1.0, ls='--', label='$E_F$')
for pos in hs_pos:
    ax.axvline(pos, color='gray', lw=0.6)
ax.set_xlim(0, 1)
ax.set_ylim(-10, 8)
ax.set_xticks(hs_pos)
ax.set_xticklabels(hs_labels, fontsize=13)
ax.set_ylabel('Energy (eV)', fontsize=13)
ax.set_title('Band structure (swap in real QE data above)', fontsize=11)
ax.legend(fontsize=11)
plt.tight_layout()
plt.savefig('sim3_band_structure.pdf', dpi=150)
plt.show()

filled = bands[bands < 0]
empty = bands[bands > 0]
if len(filled) > 0 and len(empty) > 0:
    print(f"Estimated band gap: {empty.min() - filled.max():.3f} eV")
```

Key Takeaway

The block marked with dashes is the only part that changes when real data arrives. Learn the plotting on fake data so you are not debugging two things at once when you finally run QE.

Simulation 4: Density of States by Hand

What is this about? The density of states answers one question: at each energy level, how many electron states are available? We build it from scratch by placing a small Gaussian bump at each allowed energy and adding them all up. This is exactly what real DFT codes do internally.

What to look for: After you see the gap in the DOS, change `sigma` from 0.15 to 1.5 and run again. The gap disappears. The physics has not changed. The smearing is hiding it.

This is why the smearing width in every DOS plot matters.

sim4_density_of_states.py

```
import numpy as np
import matplotlib.pyplot as plt

# Fake eigenvalues from a pretend DFT calculation.
# In reality, QE gives you thousands of these from many k-points.
np.random.seed(42)
valence = np.random.normal(loc=-3.0, scale=2.0, size=2000)
conduct = np.random.normal(loc= 2.0, scale=1.5, size=2000)
eigenvalues = np.concatenate([valence, conduct])

# Cut out the artificial gap between -0.5 and +0.8 eV
eigenvalues = eigenvalues[(eigenvalues < -0.5) | (eigenvalues > 0.8)]

# Build the DOS: each eigenvalue becomes a Gaussian bump
E_grid = np.linspace(-10, 8, 1000)
sigma = 0.15 # smearing width in eV. Try 0.5, then 1.5!
dos = np.zeros_like(E_grid)
for e in eigenvalues:
    dos += np.exp(-0.5 * ((E_grid - e) / sigma)**2)
dos /= (sigma * np.sqrt(2 * np.pi))

# Plot
fig, ax = plt.subplots(figsize=(5, 7))
ax.plot(dos, E_grid, color='darkorange', lw=2)
ax.fill_betweenx(E_grid, dos, where=(E_grid < 0),
                color='darkorange', alpha=0.25, label='Filled states')
ax.axhline(0, color='black', lw=1.0, ls='--', label='$E_F$')
ax.set_xlabel('DOS (states / eV)')
ax.set_ylabel('Energy (eV)')
ax.set_title(f'Density of States (sigma = {sigma} eV)')
ax.set_xlim(left=0)
ax.set_ylim(-10, 8)
ax.legend()
plt.tight_layout()
plt.savefig('sim4_dos.pdf', dpi=150)
plt.show()

gap = E_grid[(dos < 0.5) & (E_grid > -1) & (E_grid < 3)]
if len(gap) > 0:
    print(f"Gap: {gap[0]:.2f} to {gap[-1]:.2f} eV")
    print(f"Estimated band gap: {gap[-1] - gap[0]:.2f} eV")
```

Intuition

Change `sigma` from 0.15 to 1.5 and run again. The gap disappears. The physics has not changed at all. The smearing just blurs over it. This is why you always state the smearing width when you show a DOS plot. Any DOS that shows a clean gap without mentioning smearing should be read with suspicion.

11 Setting Up Quantum ESPRESSO

Historical Insight

Quantum ESPRESSO is free and open-source. The input files are plain text that you write yourself. Every parameter has to be named explicitly. There is no GUI to hide behind.

That is not a disadvantage. That is what makes it good for learning. When

something goes wrong, and something will always go wrong, you know exactly which line to look at.

Installation

Install Quantum ESPRESSO via conda

```
conda create -n qe_env python=3.10
conda activate qe_env
conda install -c conda-forge qe
```

Check it worked

```
pw.x --version
```

Install Python post-processing tools

```
pip install ase pymatgen seekpath
```

Key Takeaway

`pw.x` is the core program. It runs the SCF loop, handles structural relaxation, and computes band structures. All other programs (`bands.x`, `dos.x`, `pp.x`) just read what `pw.x` already produced.

Pseudopotentials

Pseudopotentials replace the tightly-bound core electrons (the ones deep inside each atom that never participate in bonding) with a smooth effective potential. You only compute the valence electrons, which are the ones that actually do something interesting.

Download the Si pseudopotential

```
mkdir -p ~/pseudos && cd ~/pseudos
wget https://pseudopotentials.quantum-espresso.org/upf_files/Si.pbe-n-rrkjus_psl.1.0.0.UPF
```

Intuition

Think of the pseudopotential as the structural drawings for the basement of a building. You replace a very detailed description of the core (core electrons) with a simple note: “basement: load-bearing, do not touch.” The floors you actually care about (valence electrons) are what you work with.

The SCF Input File for Silicon

si_scf.in – annotated input file

```
&CONTROL
  calculation = 'scf'      ! Run an SCF calculation
  prefix      = 'silicon'  ! Label for all output files
  outdir      = './out'    ! Folder for output
  pseudo_dir  = '~/pseudos'
/
&SYSTEM
  ibrav       = 2          ! Crystal type: 2 = FCC
  cellldm(1)  = 10.263     ! Lattice constant in Bohr (~5.43 Angstrom)
  nat         = 2          ! Number of atoms in the unit cell
  ntyp        = 1          ! Number of different atom types
  ecutwfc     = 40.0       ! Plane-wave cutoff in Rydberg
  ecutrho     = 320.0      ! Charge density cutoff = 8 x ecutwfc
/
&ELECTRONS
  conv_thr    = 1.0d-8     ! Stop when energy changes less than this
  mixing_beta = 0.7        ! How fast to update the density each step
/
ATOMIC_SPECIES
  Si 28.086 Si.pbe-n-rrkjus_psl.1.0.0.UPF
ATOMIC_POSITIONS {crystal}
  Si 0.00 0.00 0.00
  Si 0.25 0.25 0.25
K_POINTS {automatic}
  8 8 8 0 0 0
```

Run it

```
pw.x < si_scf.in > si_scf.out
```

Key Takeaway

The rule `ecutrho = 8 x ecutwfc` holds for ultrasoft pseudopotentials. Always run a convergence test on `ecutwfc` before trusting any result. The Python convergence test from Simulation 1 shows you exactly how to do this.

The Band Structure Input File

Once the SCF is done, you keep the charge density fixed and compute eigenvalues along the high-symmetry k-path.

si_bands.in

```
&CONTROL
  calculation = 'bands'
  prefix      = 'silicon' outdir = './out' pseudo_dir = '~/pseudos'
/
&SYSTEM
  ibrav = 2 cellldm(1) = 10.263 nat = 2 ntyp = 1
  ecutwfc = 40.0 nbnd = 8
/
&ELECTRONS
  conv_thr = 1.0d-8
/
ATOMIC_SPECIES
  Si 28.086 Si.pbe-n-rrkjus_psl.1.0.0.UPF
```

```

ATOMIC_POSITIONS {crystal}
  Si  0.00  0.00  0.00
  Si  0.25  0.25  0.25
K_POINTS {crystal_b}
5
0.500 0.500 0.500  20  ! L
0.000 0.000 0.000  30  ! Gamma
0.500 0.000 0.500  10  ! X
0.500 0.250 0.750  20  ! W
0.375 0.375 0.750  20  ! K

```

Run it and post-process

```

pw.x < si_bands.in > si_bands.out

echo "&BANDS
prefix='silicon' outdir='./out' filband='si_bands.dat'
/" | bands.x > postprocess.out

```

Intuition

You now have a file called `si_bands.dat` with real eigenvalues from an actual DFT calculation. Simulation 3 was built to plot exactly this. Swap in the real data at the top of that script, set the Fermi energy from your SCF output, and run it. That is the full loop from theory to a band structure plot you can put in a report.

12 Closing Note

Historical Insight

Walter Kohn received the Nobel Prize in Chemistry in 1998. Not Physics. Chemistry.

The committee recognised that DFT had changed how chemists, biologists, and materials scientists work, not just condensed matter physicists.

The theory that started as a two-page proof about electron density is now behind materials databases with millions of compounds, autonomous labs that synthesise new materials overnight, and transistors so small that quantum mechanics is not a small correction but the main event.

What you are teaching is not a niche computational method. It is how materials science is done today.

Three things to leave students with:

- DFT is exact in principle and approximate in practice. The only approximation is V_{xc} . Understanding its limits is not optional.
- Band diagrams are not abstract plots. They are maps of what electrons are physically allowed to do in a material. Read them like a map.
- Every device you can name, transistor, LED, solar cell, laser, is fundamentally just an engineered band diagram.

Key Takeaway

Planck's constant never left the building.

The goal of this course is to make that sentence feel obvious.

ज्ञान